

INSuRE Spring 2024
Final Report - May 3, 2024

Assessing the Potential Impact of Memory-Safe Languages for System Software Security

Investigators:

**The University of Arizona and The University of Texas at Dallas
Team 3**

Chia-Hao Yang - Student Researcher - cxy200019@utdallas.edu

Josh McCoy - Student Researcher - mccoyja3@arizona.edu

Nate Schultz - Student Researcher - n8r@arizona.edu

Ryan Brockinton - Student Researcher - rtb180002@utdallas.edu

Bill Hayes - Faculty Support - billhayes@arizona.edu

Brian Ricks - Faculty Support - bwr031000@utdallas.edu

Problem Mentor:

Neal Ziring - National Security Agency - nlzirin@uwe.nsa.gov

Table of Contents

Table of Contents	1
1.0 Executive Summary	2
2.0 Introduction	3
2.1 Top-Level Research Project Description/ Problem Statement	3
2.2 Purpose Statement	3
2.3 Motivation	5
3.0 Literature Review Summary	6
4.0 Methods and Procedures	6
4.1 Characterization of Methods Used	6
4.2 Procedures / Plans	7
4.3 Schedule	9
4.4 Deliverables	10
4.5 Limitations and Delimitations	11
5.0 Findings Overview	12
5.1 Findings Overview	12
5.2 Detailed Findings	14
5.2.1 Research Tools	14
5.2.2 Vulnerabilities in C/C++ Native and WebAssembly Environments	15
5.2.3 Effectiveness of Memory-Safe Languages (Rust and Go)	17
5.2.4 Comparative Analysis of ChatGPT 3.5 and ChatGPT 4.0 in Code Vulnerability Identification and Mitigation	24
6.0 Issues and Challenges	29
7.0 Conclusions and Recommendations	30
7.1 Conclusions	30
7.2 Recommendations	30
7.3 Conclusions and Recommendations for Comparing Chat GPT 3.5 and ChatGPT4.0	31
8.0 Future Work	32
9.0 References / Bibliography	33
10.0 Team Biographies / Tasking	34
Appendix A: Source Code on GitHub	35
Appendix B: Acronyms and Terms/ Definitions	35

1.0 Executive Summary

The relentless advancement of web technologies and the increasing complexity of web applications have underscored the importance of robust cybersecurity measures, particularly in the realm of memory safety. Traditional programming languages like C and C++ have long been the backbone of system and application development. However, they are notoriously susceptible to memory vulnerabilities, which can lead to significant security breaches. This research paper aims to provide a comprehensive assessment of these traditional memory vulnerabilities in both native C/C++ environments and the emerging platform of WebAssembly—a binary instruction format for a stack-based virtual machine, aimed at enabling high-performance applications on the web.

Simultaneously to this, the exploration into the memory safety features of modern programming languages, Rust and Go, forms a cornerstone of our study. These languages have been designed with built-in mechanisms to mitigate common memory safety issues, promising a safer development environment. Through a thorough identification of vulnerabilities via Common Weakness Enumeration (CWE) analysis, this paper aims to bridge the gap between theoretical knowledge and practical application, providing coding examples in multiple languages to illustrate the nuances of memory safety across different programming environments.

A significant portion of our research is devoted to testing these vulnerabilities within the WebAssembly environment, with a particular focus on assessing the memory safety enhancements offered by Rust and Go. This investigation is not only pivotal for understanding the security implications of deploying WebAssembly applications but also for evaluating the potential of these modern languages in elevating the security posture of web applications at large.

This paper also explores the integration of Artificial Intelligence (AI) tools, notably ChatGPT, in the process of vulnerability detection and remediation. The effectiveness of these AI-powered tools in parsing and analyzing source code for potential security vulnerabilities offers a promising avenue for enhancing the security of software development workflows. By automating the detection and mitigation of memory vulnerabilities, AI tools have the potential to significantly reduce the incidence of exploitable security flaws in software applications.

This research is driven by the goal of providing a holistic overview of the current landscape of memory vulnerabilities in software development, with a keen eye on the future of memory safety in web technologies. Through the provision of empirical evidence and practical insights, this paper aims to contribute valuable knowledge to the fields of cybersecurity and software development, empowering developers, security professionals, and researchers with the tools and understanding necessary to forge safer, more secure software for the future.

2.0 Introduction

2.1 Top-Level Research Project Description

This project assesses the effectiveness of traditional C/C++ memory mitigations in WebAssembly and explores the memory safety features of Rust and Go in this context. By identifying vulnerabilities through CWE analysis, coding examples in multiple languages, and testing in WebAssembly, the project aims to determine if Rust and Go provide superior memory safety. Integration of AI tools like ChatGPT will further evaluate their effectiveness in detecting and remediating vulnerabilities in the source code.

2.2 Purpose Statement

In the evolving landscape of web technologies, the security of WebAssembly applications has emerged as a pivotal concern. This project report details our comprehensive examination of the viability of conventional migration strategies to address vulnerabilities inherent in WebAssembly. Simultaneously, it highlights the security advantages provided by memory-safe programming languages such as Rust and Go. Through the inclusion of real-world case studies, we illustrate the potential for successful vulnerability mitigation and provide a well-rounded perspective on enhancing security in WebAssembly development.

- **Examination of Migration Strategies**

Our research began with an in-depth examination of conventional migration strategies traditionally used to address vulnerabilities in software development. We evaluated these strategies within the unique context of WebAssembly, assessing their effectiveness in mitigating security risks. This evaluation aimed to understand how traditional approaches could be adapted

or evolved to better suit the needs of WebAssembly security.

- **Emphasis on Rust and Go**

A significant portion of our study was devoted to exploring the security benefits of Rust and Go. These modern programming languages are designed with memory safety as a core principle, offering built-in mechanisms to prevent common vulnerabilities. By analyzing the features and security models of Rust and Go, we shed light on how these languages can serve as powerful tools in enhancing the security posture of WebAssembly applications.

- **Successful Vulnerability Mitigation**

Our findings demonstrate successful examples of vulnerability mitigation within the WebAssembly context. Through a detailed analysis of specific vulnerabilities and the strategies employed to address them, we highlight the effectiveness of both conventional migration strategies and the adoption of Rust and Go. These success stories underscore the feasibility of enhancing security in WebAssembly through thoughtful application of proven and innovative methods.

- **Key Insights and Recommendations**

The culmination of our research is presented in a set of key insights and recommendations aimed at developers, security professionals, and decision-makers involved in WebAssembly development. We advocate for the adoption of memory-safe languages, such as Rust and Go, in projects where security is of paramount importance. Furthermore, our recommendations include guidance on integrating conventional migration strategies with the unique aspects of WebAssembly to achieve optimal security outcomes.

- **Advocacy for Memory-Safe Languages**

Throughout this report, we strongly advocate for the adoption of memory-safe programming languages in security-sensitive projects. The evidence gathered through our research supports the conclusion that Rust and Go can play a critical role in minimizing vulnerabilities and enhancing the overall security of WebAssembly applications.

- **Conclusion**

Our aim has been to provide a comprehensive and nuanced perspective on the challenges and solutions related to security in WebAssembly development. By examining conventional migration strategies, highlighting the benefits of Rust and Go, and presenting real-world case studies, we offer valuable insights and practical recommendations for improving security practices. As WebAssembly continues to grow in popularity, the adoption of memory-safe languages and effective migration strategies will be key to safeguarding the future of web application security.

2.3 Motivation

- **Recognition of Growing Security Concerns:** There is increasing relevance of web technologies and the inherent security risks, particularly concerning memory safety in system software.
- **Deficiencies of Traditional Languages:** The major driver for this project is that vulnerabilities associated with widely used programming languages like C and C++ are prevalent. These languages' susceptibility to memory-related security issues are a major factor in cybersecurity breaches in today's landscape.
- **Shift Towards Memory-Safe Languages:** The emergence of modern, memory-safe programming languages like Rust and Go among others is a promising solution to address longstanding security vulnerabilities.
- **WebAssembly's Role:** There is an expanding adoption of WebAssembly which could be concerning as it has potential vulnerabilities. There is a large need for a comprehensive study in this area given its use on 92% of worldwide browsers without much public awareness about it.
- **Innovation Through AI Tools:** The potential of integrating AI tools, such as ChatGPT, in enhancing software security through automated vulnerability detection and mitigation was also a topic that we really wanted to explore.

3.0 Literature Review Summary

- Existing Research on Memory Vulnerabilities: There is an extensive amount of existing literature on C/C++ memory vulnerabilities and many ongoing research efforts.
- Insights into WebAssembly Security: The main paper from our Literature Review was “Everything Old is New Again: Binary Security of WebAssembly” which focused on WebAssembly's security model, emphasizing the relatively novel exploration of its memory safety compared to traditional environments.
- Gap in Literature: There is a research gap in comparing memory safety across different programming languages and environments, particularly between traditional languages (C/C++) and memory-safe languages (Rust, Go) in the context of WebAssembly.
- Contributions of Memory-Safe Languages: Lots of new literature exists that supports the effectiveness of Rust and Go in mitigating common memory vulnerabilities, providing a theoretical basis for their potential security benefits.
- Role of AI in Security: Studies examining the use of AI and machine learning tools in identifying and rectifying security vulnerabilities are becoming more prevalent. This suggests a growing trend towards automation in cybersecurity.

4.0 Methods and Procedures

4.1 Characterization of Methods Used

- Selection of Common Weakness Enumerations (CWEs): We utilized CWEs website as our main method of identifying and selecting relevant CWEs that are applicable to the study of memory vulnerabilities in WebAssembly and traditional programming environments.
- Development of Test Cases: We worked towards creating C/C++ examples that demonstrate specific vulnerabilities in both native and WebAssembly contexts, intending to illustrate the practical implications of theoretical vulnerabilities.
- Analysis of WebAssembly Vulnerabilities: We utilized research methods to study documented vulnerabilities specific to WebAssembly, including an examination of cases listed in the National Vulnerability Database (NVD).

- **Migration Strategies and Mitigation Techniques:** We examined traditional mitigation strategies for memory vulnerabilities in native environments and their applicability to WebAssembly, including testing their effectiveness.
- **Integration and Testing of AI Tools:** A methodological approach was used to evaluate the use of AI tools like ChatGPT in detecting and mitigating vulnerabilities within the developed test cases.
- **Comparative Analysis:** We made conclusions via a comparative study conducted between memory vulnerabilities in traditional languages (C/C++) and memory-safe languages (Rust, Go), with a focus on their manifestation and mitigation in WebAssembly environments.

4.2 Procedures / Plans

In the course of our research into memory Common Weakness Enumerations (CWEs) relevant to WebAssembly, we have made significant strides in identifying and cataloging vulnerabilities that are particularly relevant to our study. Given the expansive landscape of vulnerabilities associated with C and C++ programming languages, our initial task was to discern which of these vulnerabilities are also applicable to WebAssembly environments. This process was crucial, as not all vulnerabilities in C and C++ translate directly to WebAssembly due to its unique execution model and security features.

Identification of Relevant CWEs: Through analysis, our team has refined the list of potential vulnerabilities down to five key CWEs that are applicable and significant to our research on WebAssembly. This focus enables us to conduct a more targeted and in-depth investigation into these vulnerabilities, assessing their impact within WebAssembly contexts and exploring effective mitigation strategies.

The identified CWEs are as follows:

CWE-476: NULL Pointer Dereference - This vulnerability occurs when the application dereferences a pointer that it expects to be valid but is NULL, leading to a crash or unintended behavior.

CWE-119: Improper Restriction of Operations within the Bounds of a Memory Buffer - This category of vulnerability encompasses errors that occur when operations

are performed on a memory buffer, but the operations do not properly respect the buffer's boundaries, potentially leading to unauthorized access or information disclosure.

CWE-121: Stack-based Buffer Overflow - A specific instance of buffer overflow occurring on the stack, this vulnerability can lead to arbitrary code execution, denial of service, or escalation of privileges.

CWE-122: Heap-based Buffer Overflow - Similar to the stack-based buffer overflow, but occurring on the heap, this vulnerability can also lead to serious security breaches such as arbitrary code execution.

Hierarchical Relationship Among Identified CWEs: Of particular interest is the hierarchical relationship observed among the identified CWEs. Two of the vulnerabilities, CWE-121 and CWE-122, are considered "child" vulnerabilities of the broader category represented by CWE-119. This distinction is critical for our analysis, as it highlights the nuanced differences in how buffer overflow vulnerabilities manifest in stack and heap memory and their relevance to WebAssembly's security model.

Conclusion: The progress to date in identifying and categorizing the CWEs applicable to WebAssembly represents a significant milestone in our research. By focusing on these five key vulnerabilities, we aim to dive deeper into the mechanics of these security issues within the WebAssembly environment, test their exploitability, and develop comprehensive mitigation strategies. This targeted approach not only streamlines our research efforts but also enhances the potential impact of our findings on the security of WebAssembly applications.

Division of Tasks: The research team is split into two primary groups: the programming team and the research team. The programming team focuses on crafting C/C++ examples that demonstrate exploitable vulnerabilities in both WebAssembly and native environments. Concurrently, the research team undertakes a comprehensive study of WebAssembly-related vulnerabilities as documented in the National Vulnerability Database (NVD), providing a theoretical foundation for the practical experiments.

4.3 Schedule

January 23, 2024	Project Bids
January 26, 2024	Project assignments
February 4, 2024	Project proposal
February 5 - February 12, 2024	Research memory CWE's that happen in WebAssembly
February 13 - February 26, 2024	Implement C/C++ examples exploiting in native environment and WebAssembly
February 26 - March 11, 2024	Implement traditional migration in native environment
March 11 - March 21, 2024	Testing traditional migration availability in WebAssembly
March 10- March 21, 2024	Midterm report writing and presentation practice run
March 22, 2024	Midterm presentations / Midterm deliverables
March 23 - April 8, 2024	Recreate C/C++ code in Rust and Go
April 9 - April 25, 2024	Testing Rust and Go code behavior in WebAssembly
April 26, 2024	Final report writing and presentation practice run
May 3, 2024	Final presentations / Project deliverables

4.4 Deliverables

This segment of the report delineates the deliverables produced over the course of our research into memory Common Weakness Enumerations (CWEs) in WebAssembly. Our project was designed to not only advance the understanding of security vulnerabilities within this domain but also to provide tangible tools and resources to the academic and industry communities for further exploration and application. Below is an overview of the key deliverables resulting from this project:

- **Comprehensive Final Report**

The cornerstone of our project deliverables is a comprehensive final report that encapsulates the entirety of our research journey. This document is meticulously structured to cover the research methodologies employed, detailed findings from our investigations, and the derived insights. It provides an exhaustive narrative of our exploration into memory CWEs in WebAssembly and their implications in both theoretical and practical contexts.

- **Professionally Crafted Presentation Materials**

To effectively disseminate our findings and insights to both academic and industry audiences, we have developed a suite of professionally crafted presentation materials. These materials are designed to convey the complexities and nuances of our research in an accessible and engaging manner, facilitating knowledge transfer and stimulating discussion among scholars, developers, and cybersecurity practitioners.

- **Dedicated Repository**

Understanding the importance of transparency and reproducibility in research, we have established a dedicated repository to house all project-related materials. This repository serves as a central hub for accessing our research methodologies, findings, presentation materials, and any supplementary documents or resources.

- **Curated Collection of Test Cases**

Within the dedicated repository, we have included a curated collection of test cases that were instrumental in our experimental investigations. These test cases are provided with comprehensive documentation to facilitate replication of experiments and validation of findings.

by others in the field. This resource aims to support ongoing research and development efforts in understanding and mitigating memory CWEs in WebAssembly.

- **Analysis Tools**

Another significant contribution to the repository is the suite of analysis tools developed over the course of our project. These tools, which were integral to our research processes, are made available for public use. They enable scholars, developers, and cybersecurity practitioners to conduct their own analyses, potentially extending the scope of our initial research or applying the tools in new and innovative contexts.

The overarching aim of these deliverables is to empower a broad spectrum of stakeholders, from scholars and developers to cybersecurity practitioners, with robust resources that not only validate the outcomes of our project but also provide a foundation for building upon these results. Through the dissemination of our final report, presentation materials, and the provisioning of a dedicated repository with test cases and analysis tools, we aspire to make a lasting contribution to the field of cybersecurity, specifically in the domain of WebAssembly security vulnerabilities.

4.5 Limitations and Delimitations

- **Limitations**

Our first limitation was the Scope of Programming Languages. Focusing on specific programming languages (C/C++, Rust, Go) and WebAssembly was critical to our project but at times could lead to finding a vulnerability in a bigger language like C or C++ that was not applicable to the newer/safer languages. The next limitation was the depth of vulnerability analysis. There were at times limitations in how deep of information we are able to find on a given Common Weakness Enumeration (CWE) for our specific use cases and languages of choice. A third limitation of our testing was testing environmental constraints. By utilizing testing environments for evaluating vulnerabilities and mitigation strategies we may not have fully replicated real-world operational conditions. Our last limitation was AI Tool Effectiveness. Limitations exist related to the evaluation of AI tools for vulnerability detection and mitigation, including

potential biases in tool performance. This may have led to a more constrained scope of vulnerabilities they can effectively address.

- **Delimitations**

The first delimitation was the focused scope on memory safety. The research specifically delimits its scope to the study of memory safety vulnerabilities, excluding other types of security issues which allowed for a more targeted investigation. Our second delimitation was the selection of WebAssembly as a case study. The deliberate choice to focus on WebAssembly, given its emerging importance and unique security considerations, was a delimitation to narrow the research focus. Next was our choice of memory-safe languages. The rationale behind selecting Rust and Go for the study was due to their designed-in memory safety features, delimiting the research to languages with inherent mechanisms to prevent memory vulnerabilities. Another delimitation was the use of specific AI tools. The selection of particular AI tools (e.g., ChatGPT) for vulnerability analysis, delimited the study to tools that were accessible and deemed most relevant for the research objectives. Our last delimitation was our empirical and theoretical approach. The research's delimitation to a combination of empirical testing and theoretical analysis, aimed to balance practical findings with conceptual insights into memory safety issues.

5.0 Findings

5.1 Findings Overview

In our literature research and review we have found a few things. Extensive literature exists on C/C++ memory vulnerabilities and surprisingly there is also a good amount of literature on WebAssembly memory vulnerabilities. Literature reviews and research have shown our desire to test WebAssembly alongside C/C++ compared to “Memory Safe” languages is a quite worthy project. WebAssembly is implemented in 92% of worldwide browsers and thus is being used by many users who don’t even realize it. Some CWE’s we found are not applicable in WebAssembly (even if they are in C or C++) so this led us to tailor our vulnerability selection to those we could test in both WebAssembly and C languages.

- **CWE-121: Stack-based Buffer Overflow:**

WebAssembly lacks stack overflow detection compared to native environments, leading to potential data corruption risks. EMCC-compiled binaries may allow stack buffers declared later to overflow into earlier ones. Additionally, in WebAssembly, binary code is cleared before allocating memory to the stack section, unlike in native environments where uncleared binary code may remain.

- **CWE-122: Heap-based Buffer Overflow:**

Compared to native environments, WebAssembly lacks effective mitigations against heap overflows, making it more vulnerable to corresponding attacks. GCC-compiled binaries offer protections like memory address space layout randomization and a 416-byte gap between heap buffers, significantly reducing the risk of overflow. However, in WebAssembly, heap buffer locations are fixed during execution with only a 16-byte gap, allowing simple overflow attacks to remain effective.

- **CWE-476: NULL Pointer Dereference:**

Compared to native environments, WebAssembly contains the same flaws against null pointer dereferences. Both environments have difficulties detecting the faults at compile time which can lead to crashes or undefined behaviors of the program at runtime. Although the risk of program crashes is significant and apparent, the risk of the visible success or undefined behaviors of the program can remain unknown. For this fault, WebAssembly does not provide any more protections or mitigations than the native environment creating the same risk for the faults to occur.

5.2 Detailed Findings

5.2.1 Research Tools

- **WebAssembly Environment Testing**

The testing compiler used in the native environment for C code is GNU Compiler Collection (GCC); The testing compiler used in WebAssembly environment is Emscripten Compiler Frontend (EMCC).

- **Memory Dynamic Analysis Tool: C/C++ DevTools Support (DWARF)**

A DevTools Plugin for debugging C/C++ WebAssembly applications (using DWARF debug information), provided by Chrome DevTools Team. It helps to trace memory usage in browsers like GDB in the native environment.

5.2.2 Vulnerabilities in C/C++ Native and WebAssembly Environments

- **CWE-121: Stack-based Buffer Overflow**

Compared to the native environment, the WebAssembly environment lacks detection of stack overflows. GCC-compiled binaries by default abort execution when a stack overflow is detected, while in EMCC-compiled binaries, a stack buffer declared later can overflow into a stack buffer declared earlier. This result shows that the WebAssembly environment has a corresponding risk of data corruption. As an additional research finding, we discovered that the WebAssembly environment clears the binary code before allocating memory to the stack section, whereas the native environment may leave behind uncleared binary code.

<pre> argv[1]: a_20bytes_1234567890 Execute: char buf1[16], buf2[16]; strcpy(buf1, "password"); Stack Data 1: password, Memory Address: 0x16b1b3388 Stack Data 2: 04, Memory Address: 0x16b1b3378 Execute: strcpy(buf2, argv[1]); Trace/BPT trap: 5 </pre> Native Environment	<pre> argv[1]: a_20bytes_1234567890 Execute: char buf1[16], buf2[16]; strcpy(buf1, "password"); Stack Data 1: password, Memory Address: 0xffa0 Stack Data 2: , Memory Address: 0xff90 Execute: strcpy(buf2, argv[1]); Stack Data 1: 7890, Memory Address: 0xffa0 Stack Data 2: a_20bytes_1234567890, Memory Address: 0xff90 </pre> WebAssembly Environment
--	---

- **CWE-122: Heap-based Buffer Overflow**

Compared to the native environment, the WebAssembly environment lacks corresponding mitigations against heap overflows due to memory allocation deficiencies, resulting in a relatively much higher risk of being attacked by corresponding methods. From the testing cases, it is known that in the default mode, GCC-compiled binaries provide memory address space layout randomization mitigation and a 416-byte gap between different heap buffers, greatly reducing the risk of heap buffer overflow. However, in the WebAssembly environment, the memory location of each heap buffer is fixed during execution, with only a 16-byte gap. As a result, simple heap overflow attacks that have been prevented in the native environment can still be effective in the WebAssembly environment.

<pre> argv[1]: a_20bytes_1234567890 argv[2]: b_20bytes_1234567890 Execute: char *ptr1, *ptr2; ptr1 = malloc(8); ptr2 = malloc(8); Heap Data 1: , Memory Address: 0x151605e60 Heap Data 2: , Memory Address: 0x151606000 Execute: strcpy(ptr1, argv[1]); Heap Data 1: a_20bytes_1234567890, Memory Address: 0x151605e60 Heap Data 2: , Memory Address: 0x151606000 Execute: strcpy(ptr2, argv[2]); Heap Data 1: a_20bytes_1234567890, Memory Address: 0x151605e60 Heap Data 2: b_20bytes_1234567890, Memory Address: 0x151606000 </pre> Native Environment	<pre> argv[1]: a_20bytes_1234567890 argv[2]: b_20bytes_1234567890 Execute: char *ptr1, *ptr2; ptr1 = malloc(8); ptr2 = malloc(8); Heap Data 1: , Memory Address: 0x10a88 Heap Data 2: , Memory Address: 0x10a98 Execute: strcpy(ptr1, argv[1]); Heap Data 1: a_20bytes_1234567890, Memory Address: 0x10a88 Heap Data 2: 7890, Memory Address: 0x10a98 Execute: strcpy(ptr2, argv[2]); Heap Data 1: a_20bytes_123456b_20bytes_1234567890, Memory Address: 0x10a88 Heap Data 2: b_20bytes_1234567890, Memory Address: 0x10a98 </pre> WebAssembly Environment
--	---

- **CWE-476: NULL Pointer Dereference**

In both the native and WebAssembly environments null pointer dereferences work similarly within C. At compile time the GCC or emcc (WebAssembly compiler) doesn't present any warnings to warn of the null pointer which leads to the program crashing at runtime. Both compilers have flags, options to customize the compile process of the file, which could be used to try to detect the fault but there are test cases where these flags do not detect the fault. From the testing cases, the native environment produces a segmentation fault when the null pointer is dereferenced due to the program trying to access memory location zero which is a restricted address whereas WebAssembly aborts the program due to the same cause. On the other hand, C++'s object-oriented design allows for more undefined behaviors. When a null pointer dereference occurs on a class object a segmentation fault could occur if a function is called within the object that references a virtual function or a member variable which would require the pointer to be dereferenced but if neither a virtual function nor member variable is referenced within that called function the optimizations of the compiler bypass the dereference and could appear to be successful leading to undefined behavior.

5.2.3 Effectiveness of Memory-Safe Languages (Rust and Go)

Provide specific findings on how Rust and Go mitigate these vulnerabilities, supported by code examples and experimental results. For the native environment of Rust, we use Rustc, the official compiler; and for the webassembly environment of Rust, we use the tool "Wasm-pack". "Wasm-pack" is the most common tool to build and work with rust-generated WebAssembly interoped with JavaScript, in the browser or with Node.js. It also can help to build rust-generated WebAssembly packages that users could publish to the npm registry.

- **CWE-121: Stack-based Buffer Overflow**

- **Rust**

Rust in the native environment has randomized stack memory addresses, it would be safer than GCC which makes C executable files have fixed stack memory addresses by default. And in webassembly have fixed ones in default. The stack-based buffer overflow will cause RuntimeError in webassembly, which makes programs with hidden buffer overflow vulnerabilities unable to pass checks and execute like WebAssembly code converted from C language. If the length of the argument doesn't trigger a buffer overflow, it will execute normally.

<pre> arg: a_20bytes_1234567890 Execute: let mut buf1 = [0u8; 16]; let buf2 = [0u8; 16]; buf1[..8].copy_from_slice(b"password"); Stack Data 1: "password", Memory Address: 0x16b42afc0 Stack Data 2: "", Memory Address: 0x16b42afd0 Execute: buf1[..args[1].len()].copy_from_slice(args[1].as_bytes()); thread 'main' panicked at stackRust.rs:22:9: range end index 20 out of range for slice of length 16 note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace </pre> Rust Native Environment	<pre> arg: a_20bytes_1234567890 ✖ ▶ Uncaught (in promise) RuntimeError: unreachable at stack_rust_wasm_bg.wasm:0x4121 at stack_rust_wasm_bg.wasm:0x492c at stack_rust_wasm_bg.wasm:0x4539 at stack_rust_wasm_bg.wasm:0x190a at stack_test (stack_rust_wasm.js:143:14) at run (index.html:16:26) </pre> <pre> arg: a_16bytes_123456 Execute: let mut buf1 = [0u8; 16]; let buf2 = [0u8; 16]; buf1[..8].copy_from_slice(b"password"); Stack Data: "password", Memory Address: 0xffff98 Stack Data: "", Memory Address: 0xfffa8 Execute: buf1[..arg.len()].copy_from_slice(arg.as_bytes()); Stack Data: "a_16bytes_123456", Memory Address: 0xffff98 Stack Data: "", Memory Address: 0xfffa8 </pre> Rust WebAssembly Environment
--	---

- **Go**

In both the native environment and WebAssembly Environment stack-based buffer overflow is mitigated in Go. Due to the inherent checks built into the language overflow is not possible while using default safe functions. The example that was used in C was inserting a string into an array on the stack that has a smaller size than the string. This would cause the string to overwrite data outside of the allocated memory causing a memory overflow. This occurs because strings in C are null-terminated meaning the function strcpy() would continue writing to memory until the null terminator was found. In contrast, using the same example converted over to Go no overflow occurs. Strings in Go contain a length attribute stored with the string meaning when the string is copied over to the slice

using the function `copy()` the data would stop writing to memory as soon as the maximum size of the allocated memory of the slice is reached. In the WebAssembly environment, this is still true so the issue isn't present in this environment as well.

<pre>\$ go run stackBufferOverflow.go a_20bytes_1234567890 argv[1]: a_20bytes_1234567890 Execute: var buf1 [16]byte var buf2 [16]byte copy(buf1[:], "password") Stack Data 1: password, Memory Address: 0xc00000a110 Stack Data 2: , Memory Address: 0xc00000a120 Execute: copy(buf2[:], os.Args[1]) Stack Data 1: password, Memory Address: 0xc00000a110 Stack Data 2: a_20bytes_123456, Memory Address: 0xc00000a120</pre> Native Environment	<pre>input1: a_20bytes_1234567890 wasm_exec.js:22 Execute: wasm_exec.js:22 var buf1 [16]byte var buf2 [16]byte copy(buf1[:], "password") Stack Data 1: password , Memory Address: 0x140c050 wasm_exec.js:22 Stack Data 2: , Memory Address: 0x140c060 wasm_exec.js:22 Execute: wasm_exec.js:22 copy(buf2[:], input1) Stack Data 1: password , Memory Address: 0x140c050 wasm_exec.js:22 Stack Data 2: a_20bytes_123456, Memory Address: 0x140c060 wasm_exec.js:22</pre> WebAssembly Environment
--	---

• CWE-122: Heap-based Buffer Overflow

○ Rust

Rust in the native environment has randomized heap memory addresses, while in WebAssembly has fixed ones in default. The heap-based buffer overflow will cause `RuntimeError` in webassembly, which makes programs with hidden buffer overflow vulnerabilities unable to pass checks and execute like WebAssembly code converted from C language. If the length of the argument doesn't trigger buffer overflow, it will execute normally.

<pre>argv[1]: a_20bytes_1234567890 Execute: let mut buf1 = vec![0u8; 16]; let buf2 = vec![0u8; 16]; buf1[..8].copy_from_slice(b"password"); Heap Data 1: "password", Memory Address: 0x15b0041c0 Heap Data 2: "", Memory Address: 0x15b0041d0 Execute: buf1[..args[1].len()].copy_from_slice(args[1].as_bytes()); thread 'main' panicked at heapRust.rs:27:9: range end index 20 out of range for slice of length 16 note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace</pre> Rust Native Environment	<pre>arg: a_20bytes_1234567890 ✖ Uncaught (in promise) RuntimeError: unreachable at heap_rust_wasm_bg.wasm:0x4126 at heap_rust_wasm_bg.wasm:0x49d5 at heap_rust_wasm_bg.wasm:0x453e at heap_rust_wasm_bg.wasm:0x190f at heap_test (heap_rust_wasm.js:143:14) at run (index.html:16:26)</pre>
---	--

	<pre> arg: a_16bytes_123456 Execute: let mut buf1 = vec![0u8; 16]; let buf2 = vec![0u8; 16]; buf1[..8].copy_from_slice(b"password"); Stack Data: "password", Memory Address: 0x110020 Stack Data: "", Memory Address: 0x110038 Execute: buf1[..arg.len()].copy_from_slice(arg.as_bytes()); Stack Data: "a_16bytes_123456", Memory Address: 0x110020 Stack Data: "", Memory Address: 0x110038 </pre> Rust WebAssembly Environment
--	--

○ Go

In both the native environment and WebAssembly environment heap-based buffer overflow is mitigated in Go. Due to the inherent checks built into the language overflow is not possible while using default safe functions. The example that was used in C was inserting a string into an array on the heap that has a smaller size than the string. This would cause the string to overwrite data outside of the allocated memory causing a memory overflow. This occurs because strings in C are null-terminated meaning the function strcpy() would continue writing to memory until the null terminator was found. In contrast, using the same example converted over to Go no overflow occurs. Strings in Go contains a length attribute stored with the string meaning when the string is copied over to the slice using the function copy() the data would stop writing to memory as soon as the maximum size of the allocated memory of the slice is reached. In the WebAssembly environment, this is still true so the issue isn't present in this environment as well.

<pre> \$ go run heapBufferOverflow.go a_20bytes_1234567890 b_20bytes_1234567890 argv[1]: a_20bytes_1234567890 argv[2]: b_20bytes_1234567890 Execute: var ptr1, ptr2 []byte ptr1 = make([]byte, 8) ptr2 = make([]byte, 8) Heap Data 1: , Memory Address: 0xc0000a0f8 Heap Data 2: , Memory Address: 0xc0000a130 Execute: copy(ptr1, os.Args[1]) Heap Data 1: a_20byte, Memory Address: 0xc0000a0f8 Heap Data 2: , Memory Address: 0xc0000a130 Execute: copy(ptr2, os.Args[2]) Heap Data 1: a_20byte, Memory Address: 0xc0000a0f8 Heap Data 2: b_20byte, Memory Address: 0xc0000a130 </pre>	<pre> input1: a_20bytes_1234567890 wasm_exec.js:22 input2: b_20bytes_1234567890 wasm_exec.js:22 Execute: var ptr1, ptr2 []byte ptr1 = make([]byte, 8) ptr2 = make([]byte, 8) Heap Data 1: , Memory Address: 0x140c0b0 wasm_exec.js:22 Heap Data 2: , Memory Address: 0x140c0b8 wasm_exec.js:22 Execute: copy(ptr1, input1) Heap Data 1: a_20byte, Memory Address: 0x140c0b0 wasm_exec.js:22 Heap Data 2: , Memory Address: 0x140c0b8 wasm_exec.js:22 Execute: copy(ptr2, input2) Heap Data 1: a_20byte, Memory Address: 0x140c0b0 wasm_exec.js:22 Heap Data 2: b_20byte, Memory Address: 0x140c0b8 wasm_exec.js:22 </pre>
---	---

Native Environment	WebAssembly Environment
--------------------	-------------------------

- **CWE-476: NULL Pointer Dereference**

- **Rust**

In this CWE, we test both “Safe” and “Unsafe” Rust. Safe Rust is the general rule for coding, while there is another way to manipulate memory-related actions called “Unsafe” Rust. Unsafe Rust exists because, by nature, static analysis is conservative. Rust needs to allow users to do low-level systems programming, such as directly interacting with the operating system or even writing an operating system. There are two ways to use unsafe Rust. The first and safer way is to declare an unsafe function and call in the unsafe block (`unsafe { }`), which will get an error if not called in the unsafe block. The second way is to declare a safe function but have unsafe code inside, which will not get an error if not called in the unsafe block. The second approach introduces a potential issue where developers might inadvertently utilize Unsafe Rust code when calling a black-box Rust function.

- **Safe Rust:**

We use a reference pointer to test the null pointer difference, which borrows a value without owning it. The borrow rule prohibits rewriting the address of the pointer after initiation, so it will not make a null pointer difference happen.

- **Unsafe Rust:**

We use a raw pointer to test the null pointer difference, which directly manipulates memory addresses and bypasses Rust's borrowing rules. The address of pointers may be changed to null. In the native environment, it will cause a segmentation fault; and in the WebAssembly environment, it will return 0 when meeting the null pointer.

<pre>safe_rust_function: Address of ref_pointer: 0x16d7830bc Value of ref_pointer: 10 unsafe_rust_function: Address of raw_pointer: 0x16d7830bc Value of raw_pointer: 10 Execute: raw_pointer = std::ptr::null_mut(); Address of raw_pointer: 0x0 zsh: segmentation fault ./nullPointer</pre> Rust Native Environment	<pre>safe_rust_function ----- Address of ref_pointer: 0xffffa4 Value of ref_pointer: 10 unsafe_rust_function ----- Address of raw_pointer: 0xffff8c Value of ref_pointer: 10 Execute: raw_pointer = std::ptr::null_mut(); Address of raw_pointer: 0x0 Value of ref_pointer: 0</pre> Rust WebAssembly Environment
---	--

Go

Both the native environment and WebAssembly environment mitigate null pointer dereferences in Go. Within the language, there are built-in checks for when a pointer is dereferenced and if that pointer is found to be nil (Go's version of null) then runtime panic occurs. This runtime panic is when the compiler detects possible undefined behavior and errors out with an error stating that a nil pointer dereference occurred. This prevents situations like the C++ example where a pointer that was originally pointing to an object is changed to null and still functioning when a function within that object is called that does not reference any local variables or virtual functions. This situation would be undefined behavior in C++ but is not allowed in Go.

<pre>\$ go run nullPointerDereference.go v panic: runtime error: invalid memory address or nil pointer dereference [signal 0xc0000005 code=0x1 addr=0x0 pc=0x4a1f5e] goroutine 1 [running]: main.main() C:/Users/rbroc/Documents/CS 6V81 INSuRE/UTD Project/CWE476/Go/nullPointerDereference.go:23 +0x5e exit status 2</pre> Native Environment	<pre>panic: runtime error: invalid memory address or nil pointer dereference wasm_exec.js:22 [signal 0xb code=0x0 addr=0x0 pc=0x0] wasm_exec.js:22 goroutine 1 [running]: wasm_exec.js:22 main.main() wasm_exec.js:22 C:/Users/rbroc/Documents/CS 6V81 INSuRE/UTD Project/CWE476/Go/WebA ssembly/nullPointerDereference.go:20 +0x2 exit code: 2 wasm_exec.js:101</pre> WebAssembly Environment
---	---

- **CWE Analysis**

- Key CWEs Applicable to WebAssembly: Summarize the characteristics and impact of the primary CWEs relevant to WebAssembly identified in the study, such as CWE-121 (Stack-based Buffer Overflow) and CWE-122 (Heap-based Buffer Overflow).
- Mitigation Strategies in Rust and Go: Discuss the built-in mechanisms within Rust and Go that address these CWEs and compare their effectiveness to traditional mitigation strategies in C/C++ in WebAssembly.

- Rust:

For the buffer overflow issues (CWE 121 and 122), Rust provided better protection versus C language in the WebAssembly environment. For the null pointer difference issue, though unsafe Rust remains the vulnerability, the safe Rust would not meet this problem by the borrowing rule of Rust language.

- Go:

Go's language and compiler provide safe functions and inbuilt checks to prevent the CWEs that were defined in this research. The structure of strings containing a length attribute instead of C's null-terminated strings and safeties like runtime panic make Go a memory-safe language.

- Limitations of Current Approaches
 - Challenges in Adopting Memory-Safe Languages: Document any identified challenges or limitations in fully leveraging the memory safety features of Rust and Go, such as compatibility issues, learning curves, or performance considerations.
 - Areas Lacking Sufficient Mitigation: Highlight specific types of vulnerabilities or attack vectors for which current memory safety mechanisms and AI tools may not provide adequate protection, suggesting areas for future research and development.

5.2.4 Comparative Analysis of ChatGPT 3.5 and ChatGPT 4.0 in Code Vulnerability Identification and Mitigation

- **Methodology**

The evaluation of ChatGPT 3.5 and ChatGPT 4.0's ability to analyze code vulnerabilities employed a focused methodology across several stages:

- **Identification of Issues:** Both versions were assessed on their ability to detect and describe specific coding vulnerabilities such as NULL pointer dereference and buffer overflows.
- **Depth of Analysis:** The depth of the discussion provided by each version on the identified issues, including potential implications and risks, was evaluated.
- **Recommendations:** The immediate code fixes and broader programming practices suggested by each version to enhance security were assessed.
- **Comparative Analysis:** A comparative analysis was conducted to summarize and highlight differences in the analytical capabilities of each version, noting improvements in the newer version.

This structured approach ensured a direct and systematic comparison of how each AI version addresses similar code challenges.

- **CWE-121 Stack-based Buffer Overflow**

- **ChatGPT 3.5 Analysis:**
 - **Identification:** Noted the vulnerability to stack-based buffer overflow due to the use of strcpy without bounds checking.
 - **Depth of Analysis:** Focused primarily on the overflow issue and its immediate consequences.
 - **Recommendation:** Suggested replacing strcpy with strncpy and provided a modified code example that includes bounds checking to prevent overflow.
- **ChatGPT 4.0 Analysis:**

- **Identification:** Identified not only the buffer overflow risk but also additional issues such as the use of uninitialized memory and missing input validation.
- **Depth of Analysis:** Offered a detailed breakdown of potential security implications, including data exposure and how these vulnerabilities could assist attackers.
- **Recommendation:** Recommended broader best practices such as implementing bounds checking, ensuring input validation, initializing memory buffers, and avoiding the exposure of memory addresses.
- **Results and Discussion**

The comparative analysis highlights significant improvements in ChatGPT 4.0 over its predecessor. While ChatGPT 3.5 provides competent identification and solutions for specific coding issues, it tends to focus narrowly on immediate fixes. ChatGPT 4.0, in contrast, demonstrates a deeper understanding of the broader context of vulnerabilities, suggesting comprehensive best practices that address multiple aspects of coding security.

This enhancement in analytical capabilities can be attributed to advancements in the underlying model architecture and training methodologies, which enable ChatGPT 4.0 to provide a more holistic analysis and robust security recommendations. These improvements are crucial for developing secure software, as they allow developers to anticipate and mitigate a broader range of potential security threats.
- **Conclusion**

The detailed comparison of ChatGPT 3.5 and ChatGPT 4.0 in handling code vulnerabilities underscores the evolution of AI-assisted programming tools. By leveraging these tools effectively, developers can enhance their code's security posture and reduce the risk of exploitable vulnerabilities. This study recommends the adoption of advanced AI models like ChatGPT 4.0 for a more comprehensive approach to securing software applications.

- **CWE-122: Heap-based Buffer Overflow**

- **Introduction**

Heap-based buffer overflows are critical security vulnerabilities that can lead to various exploits such as arbitrary code execution and system crashes. Effective identification and mitigation of such vulnerabilities are paramount for secure software development. This study employs two versions of the ChatGPT model to analyze their proficiency in handling these vulnerabilities.

- **Methodology**

Two distinct aspects were considered for each AI model:

- **Identification of Issues:** How each model identifies and explains the buffer overflow vulnerability.
- **Depth of Analysis:** The thoroughness with which each model evaluates the implications and potential risks associated with the vulnerability.
- **Recommendations:** The corrective measures and best practices each model suggests to mitigate the vulnerability.

- **Results**

- **ChatGPT 3.5:**

- **Identification:** Highlights the vulnerability due to insufficient buffer size and the use of `strcpy`.
- **Depth of Analysis:** Focuses on the occurrence of buffer overflow and its immediate consequences.
- **Recommendation:** Advises the use of `strncpy` for better bounds management and includes a code example with the modified approach.

- **ChatGPT 4.0:**

- **Identification:** Provides a comprehensive analysis including issues like heap buffer overflow, missing input validation, uninitialized memory usage, lack of error checking, and potential memory leaks.
- **Depth of Analysis:** Discusses the broader impact of these issues in real-world scenarios and how they might be exploited.
- **Recommendation:** Suggests a set of best practices such as checking input length, validating inputs, using `calloc` for memory

initialization, handling errors in malloc, and freeing allocated memory.

- **Discussion**

The comparative analysis indicates that ChatGPT 4.0 exhibits a more thorough understanding and approach to heap-based buffer overflow issues. Unlike its predecessor, ChatGPT 4.0 identifies additional issues and implications that extend beyond the immediate bugs to encompass broader programming practices and security implications.

- **Conclusion**

Both ChatGPT versions provide valuable insights and recommendations for addressing CWE-122 vulnerabilities. However, ChatGPT 4.0's recommendations are notably more detailed and cover a broader spectrum of best practices, which can significantly enhance security in real-world applications. This progression in model capabilities suggests that newer AI tools can play an increasingly effective role in supporting secure software development.

- **Future Work**

Further research is recommended to explore the capabilities of AI-assisted tools in other common vulnerabilities and to potentially integrate these tools into the development environments for real-time vulnerability assessment and mitigation guidance.

- **CWE-476: NULL Pointer Dereference**

- **Introduction**

NULL pointer dereference is a common vulnerability that occurs when a program attempts to use a pointer that has not been initialized to a memory location. This issue can lead to unexpected behavior, including program crashes or other security vulnerabilities. Effective management of this issue is crucial in secure software development.

- **Methodology**

The analysis focused on three core aspects:

- **Identification of Issues:** Evaluating how each AI model identifies the NULL pointer dereference and related issues.
- **Depth of Analysis:** Assessing the thoroughness of the analysis concerning the potential risks and broader implications.
- **Recommendations:** Reviewing the proposed solutions and best practices to mitigate the identified issues.
- **Results**
 - **ChatGPT 3.5:**
 - **Identification:** Concentrates on the potential for a NULL pointer dereference, highlighting its occurrence.
 - **Depth of Analysis:** Primarily focuses on the risk associated with dereferencing the pointer without prior NULL checks.
 - **Recommendation:** Advises on checking the pointer for NULL before use and provides a code snippet that handles memory allocation failure effectively.
 - **ChatGPT 4.0:**
 - **Identification:** Offers a broader analysis by noting both the NULL pointer dereference and incorrect memory allocation sizes.
 - **Depth of Analysis:** Extends the discussion to include implications like not accounting for the null terminator in strings, potentially leading to buffer overflow issues.
 - **Recommendation:** Recommends not only checking for NULL but also correcting memory allocation size, and stresses the importance of better error messaging and handling practices.
- **Discussion**

The comparison between ChatGPT 3.5 and ChatGPT 4.0 reveals significant enhancements in the latter's ability to analyze and suggest remedies for NULL pointer dereferences. While ChatGPT 3.5 provides basic detection and handling suggestions, ChatGPT 4.0 elaborates on the implications and integrates a broader scope of preventive measures, enhancing overall code safety and robustness.

- **Conclusion**

This analysis underlines the advancements in AI capabilities from ChatGPT 3.5 to ChatGPT 4.0, showcasing improved diagnostic and problem-solving skills that can be integral to software development processes. The recommendations by ChatGPT 4.0, encompassing a wider range of best practices, offer a more comprehensive approach to mitigating risks associated with NULL pointer dereferences.

- **Future Work**

Further investigations should aim to incorporate AI tools like ChatGPT in real-time coding environments to provide instant analysis and feedback. This integration could potentially lead to a significant reduction in common coding vulnerabilities such as CWE-476, enhancing the security and stability of software applications.

6.0 Issues and Challenges

Luckily our issues and challenges have been kept to a minimum but there are a few to list. Issues have included problems adapting software vulnerabilities into new languages, which has been seen in that some CWEs do not have the ability to be converted into WebAssembly, and the response time for our remote virtual machine has been slower than expected. We have been mitigating these issues via early testing as well as bringing in subject matter experts as needed to help. There have been more challenges for our project but these can be good for the project's development. Challenges have included: having to work through an iterative refinement of the software test cases, working to ensure the accuracy of the vulnerability comparisons, team members inexperienced with the new software languages, and the fact that some team members lack a computer science/software development background. We have been mitigating our challenges through the proper delegation of tasks to match group members backgrounds and utilizing any needed external sources to make sure we are utilizing and developing our software correctly.

7.0 Conclusions and Recommendations

7.1 Conclusions

- **Increased Memory Safety with Rust and Go:** Summarize the findings that demonstrate how memory-safe languages like Rust and Go significantly reduce common memory vulnerabilities compared to C/C++ in WebAssembly. Though the GNU Compiler Collection (GCC) has some migration against CWEs, as the C code turns into WebAssembly, it would have more vulnerabilities compared to memory-safe languages like Rust and Go.
- **Role of WebAssembly in Software Security:** Highlight the role WebAssembly plays in modern web technologies, emphasizing its security implications and the effectiveness of memory-safe languages in mitigating associated vulnerabilities.
- **Effectiveness of AI Tools:** Conclude the effectiveness of AI tools in identifying and mitigating vulnerabilities, noting any limitations observed in their capability to fully automate the security review process.
- **Challenges in Transitioning to Memory-Safe Languages:** Address the practical and technical challenges identified in adopting Rust and Go for existing projects, including learning curve, performance considerations, and compatibility issues.
- **AI tools like ChatGPT can detect the CWE based on the reasons of the surface, like strcpy() usage, but can not give the prediction of different developed environments as the result we got from the real testing program. Especially, it is relatively helpless to give accurate results regarding new research. Based on the actual code results of this project and the AI-generated content, it can be concluded that developers may take AI content as a reference but cannot fully rely on AI tools for code security at this stage.**

7.2 Recommendations

- **Adopting Memory-Safe Languages:** Strongly recommend the adoption of memory-safe languages for new projects, especially those with significant security requirements, and suggest strategies for transitioning existing codebases to these languages.

- **Integration of AI in Development Workflow:** Advise on the integration of AI tools into the software development lifecycle for continuous vulnerability scanning and mitigation, while also recommending human oversight to address AI limitations.
- **Further Research on WebAssembly Security:** Encourage continued research into the security model of WebAssembly, specifically in areas where current understanding of vulnerabilities and mitigations is limited.
- **Security Best Practices:** Advocate for the adoption of security best practices, including regular code audits, continuous education on security trends, and the implementation of security-centric development methodologies.
- **Policy and Educational Recommendations:** Suggest the development of policies that promote the use of memory-safe languages in critical software development projects. Recommend the inclusion of memory safety and the use of modern programming languages in cybersecurity and computer science curricula to prepare the next generation of developers.
- **Community Engagement and Open Source Contributions:** Encourage active participation in open source projects and security communities related to memory-safe programming and WebAssembly to foster a collaborative approach to software security.

7.3 Conclusions and Recommendations for Comparing Chat GPT 3.5 and ChatGPT4.0

- **Enhanced Identification**
 - ChatGPT 4.0 shows significantly improved capabilities over ChatGPT 3.5 in identifying a broader range of issues within code samples.
 - While ChatGPT 3.5 identifies basic errors and vulnerabilities, ChatGPT 4.0 detects more complex issues, such as uninitialized memory usage and potential security risks.
 - This enhanced capability is crucial in early stages of software development to prevent complex bugs and security vulnerabilities later.
- **Deeper Analysis**
 - ChatGPT 4.0 provides a more comprehensive analysis of identified issues than its predecessor.

- Unlike ChatGPT 3.5, which focuses on immediate and straightforward implications, ChatGPT 4.0 delves into both direct consequences and the broader implications of code vulnerabilities.
- This analysis includes detailed discussions on how such vulnerabilities could potentially be exploited, offering insights into potential impacts on overall security and functionality.
- **Recommendations**
 - ChatGPT 4.0 offers more detailed and actionable advice compared to ChatGPT 3.5.
 - Recommendations extend beyond direct code fixes to include best programming practices and preventative security measures.
 - These suggestions aim to enhance not only the immediate codebase but also to instill robust coding habits for more secure software in the future.
- **Overall Improvement**
 - The transition from ChatGPT 3.5 to ChatGPT 4.0 marks a substantial progression in AI capabilities related to software development and cybersecurity.
 - ChatGPT 4.0's refined capabilities make it a valuable tool for developers focused on improving code quality and ensuring security by design.
 - The rapid evolution of AI technologies highlights their increasing utility in complex and critical fields such as software development.
 - By leveraging advanced tools, developers and cybersecurity professionals can better identify, analyze, and address potential issues early in the development process, enhancing the efficiency, reliability, and security of software products.

8.0 Future Work

- **Testing Compiler Options:** The testing of CWEs in this report primarily utilized the compiler's default options. We look forward to further exploration of program security concerning additional compiler options in the future.
- **Addressing Performance Concerns:** Propose future research aimed at optimizing the performance of memory-safe languages to encourage wider adoption without compromising on efficiency.

- Expansion of AI Tool Capabilities: Suggest directions for enhancing the capabilities of AI tools in software security, including more advanced contextual analysis and automated remediation suggestions.
- Longitudinal Studies on Security Impact: Recommend conducting longitudinal studies to assess the long-term impact of adopting memory-safe languages and AI tools on the security posture of organizations.

9.0 References / Bibliography

Daniel Lehmann, Johannes Kinder, Michael Pradel (2020). Everything Old is New Again: Binary Security of WebAssembly. *the Proceedings of the 29th USENIX Security Symposium*, pg 217 - p 234 . Publisher. <https://www.usenix.org/system/files/sec20-lehmann.pdf>

C. Disselkoen, J. Renner, C. Watt, T. Garfinkel, A. Levy, and D. Stefan (2019). Position paper: Progressive memory safety for webassembly. *Proceedings of the 8th International Workshop on Hardware and Architectural Support for Security and Privacy*, pg 1 - pg 8. Publisher. <https://doi.org/10.1145/3337167.3337171>

Quentin Stiévenart, Coen De Roover (2021). Mohammad GhafariThe Security Risk of Lacking Compiler Protection in WebAssembly. *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*. Publisher. <https://doi.org/10.1109/QRS54544.2021.00024>

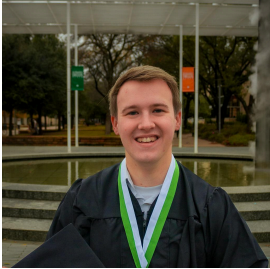
Suhyeon Song, Seonghwan Park, Donghyun Kwon (2023). metaSafer: A Technique to Detect Heap Metadata Corruption in WebAssembly. *IEEE Access*, volume: 11, pg 124887 - pg 124898. Publisher. <https://doi.org/10.1109/ACCESS.2023.3327817>

Minseo Kim, Hyerean Jang, Young-joo Shin (2022). Avengers, Assemble! Survey of WebAssembly Security Solutions. *2022 IEEE 15th International Conference on Cloud Computing*. Publisher. <https://doi.org/10.1109/CLOUD55607.2022.00077>

Marius Musch, Christian Wressnegger, Martin Johns, Konrad Rieck (). New Kid on the Web: A Study on the Prevalence of WebAssembly in the Wild. *Detection of Intrusions and Malware, and Vulnerability Assessment. DIMVA 2019. Lecture Notes in Computer Science()*, vol 11543. Springer, Cham. Publisher. https://doi.org/10.1007/978-3-030-22038-9_2

10.0 Team Biographies / Tasking

 Joshua McCoy	Education: -B.S. in Computer Science, Miami University, Oxford, Ohio. Graduate Student in Cybersecurity, University of Arizona. Experience: -Systems Security Engineer at RTX, Tucson, AZ, focusing on PIT System Cybersecurity and Anti-Tamper for missile systems. -Software Engineering intern at Microsoft (React.js). -Data Science intern at The Center for Analytics and Data Science (Data Analytics, R, RStudio). -Software Developer for The Miami Tribe of Oklahoma (HTML, CSS, JavaScript, SQL). -Associate Developer Consultant at Cyborg Mobile (React.js).
 Chia-Hao Yang	Education: -B.S. in Electrical Engineering. -M.S. in Computer Science (in progress) at the University of Texas at Dallas. -Graduate Certificate in Cyber Defense (NSA CAE). Experience: -Top 10% in NSA Codebreaker Challenge 2022. -3rd place in TexSAW 2022 CTF competition.

 Nate Schultz	Education: -B.A. in Information Communication Technology and Policy, University of Hawaii. -M.S. in Management Information Systems, University of Arizona. -Currently completing a Cybersecurity Master's at the University of Arizona. Experience: -15 years in IT, with the last 3 years as a System Administrator at RTX.
 Ryan Brockinton	Education: -B.S. in Computer Science from the University of Texas at Dallas. -Pursuing an M.S. in Computer Science with an emphasis in Cybersecurity at the same university. Experience: -Participated in Glacier CTF 2023, focusing on reverse engineering, low-level programming, and web exploitation. -Multiple internships at Texas Instruments, focusing on system automation and web development.

Appendix A: Source Code on GitHub

Source Code of Project Testcase with C, C++, Rust, and Go

<https://github.com/RyanBrockinton/Assessing-the-Potential-Impact-of-Memory-Safe-Languages-for-System-Software-Security.git>

Appendix B: Acronyms and Terms/ Definitions

Acronym	Definition
AI	Artificial Intelligence
CWE	Common Weakness Enumeration

CTF	Capture The Flag (inferred context, not explicitly defined)
EMCC	Emscripten Compiler Frontend (inferred context, not explicitly defined)
GDB	GNU Debugger
GCC	GNU Compiler Collection (inferred context, not explicitly defined)
NSA	National Security Agency
NVD	National Vulnerability Database
PIT	Platform IT
Wasm	WebAssembly

- **AI (Artificial Intelligence):** Refers to the simulation of human intelligence in machines that are programmed to think like humans and mimic their actions. AI can be applied to various sectors including software development for tasks such as vulnerability detection and remediation.
- **CWE (Common Weakness Enumeration):** A list that categorizes common software weaknesses and vulnerabilities. It serves as a standard for identifying, mitigating, and preventing software vulnerabilities.
- **CTF (Capture The Flag):** In cybersecurity, it refers to a competition where participants solve security-related challenges to find a specific piece of data, called the "flag."
- **EMCC (Emscripten Compiler Frontend):** A tool used to compile C and C++ code into WebAssembly. It allows developers to run C and C++ code on the web at near-native speed.
- **GDB (GNU Debugger):** A debugger that provides extensive facilities for tracing and altering the execution of computer programs. It's part of the GNU Project and supports various programming languages including C and C++.
- **GCC (GNU Compiler Collection):** A compiler system produced by the GNU Project supporting various programming languages. It's standard for compiling C and C++ code, among others.

- **NSA (National Security Agency):** A U.S. government agency responsible for national security and the protection of government communications and information systems against penetration and network warfare.
- **NVD (National Vulnerability Database):** A database that contains information about security-related software flaws, misconfigurations, product names, and impact metrics. It's managed by the National Institute of Standards and Technology (NIST).
- **PIT (Platform IT):** Refers to information technology solutions and systems that support specific platforms or operations, often in a specialized or embedded context.
- **Wasm/WebAssembly:** A binary instruction format for a stack-based virtual machine. WebAssembly is designed as a portable compilation target for high-level languages like C/C++ and Rust, enabling deployment on the web for client and server applications.